

Sim. Agora o prompt está suficientemente maduro para virar um plano de formação de verdade, e não apenas uma lista de tecnologias.

Eu rodei a lógica do prompt considerando três coisas simultaneamente:

- o objetivo final: Engenharia de IA, não apenas “desenvolvimento com IA”;
- o limite real de 3–4 anos: profundidade > quantidade;
- o mercado atual: sistemas de IA estão cada vez mais exigindo engenharia de software, avaliação, observabilidade, segurança e infraestrutura, não apenas modelos. Ferramentas como MLflow já tratam tracing, avaliação, monitoramento e gestão de aplicações LLM/agentes como partes do ciclo de produção. MLflow AI Platform +1

E fiz uma correção importante no próprio roadmap: Kubernetes, cloud e agentes não podem aparecer cedo demais. Eles dependem de fundamentos anteriores. A documentação oficial do Kubernetes, por exemplo, organiza o assunto em torno de workloads, networking, storage, configuração, segurança e scaling — exatamente o tipo de abstração que faz sentido depois de containers e infraestrutura básica. Kubernetes



ROADMAP FINAL — FORMAÇÃO DE ENGENHEIRO DE IA

Objetivo de saída

Ao final dos 3–4 anos, o objetivo não é:

“Davi sabe Python, LangChain, PyTorch e algumas APIs de IA.”

É:

Davi recebe um problema novo, entende o negócio, transforma o problema em requisitos, escolhe uma arquitetura, trabalha os dados, escolhe ou constrói modelos, implementa o software, mede qualidade, protege o sistema, coloca em produção e consegue mantê-lo.

A cadeia mental será:

Problema → negócio → requisitos → arquitetura → dados → algoritmo/modelo → software → avaliação → segurança → infraestrutura → deployment → observabilidade → manutenção → evolução

Essa é a espinha dorsal da formação.

REGRA DE OURO

Todo conhecimento relevante será avaliado em quatro níveis.

- **Nível 1 — Conhecer**

“Eu sei o que é.”

Consegue definir, reconhecer e identificar aplicações.

- **Nível 2 — Explicar**

“Eu sei por que existe e como funciona.”

Consegue explicar mecanismos, relações, vantagens e limitações.

- **Nível 3 — Implementar**

“Eu consigo construir.”

Consegue implementar, testar, modificar e depurar.

Documentação e IA são permitidas.

Copiar não é domínio.

- **Nível 4 — Projetar**

“Eu sei quando usar, quando não usar e quais são os trade-offs.”

Consegue comparar soluções considerando:

custo; performance; latência; escalabilidade; segurança; confiabilidade; manutenção; complexidade; observabilidade; experiência do usuário; impacto empresarial.

Distribuição do objetivo

Área	Nível-alvo
Lógica	4
Python	4
Algoritmos e estruturas de dados	4
Engenharia de software	4
Git	4
SQL	4
Backend	4
Sistemas	4
Matemática para IA	3–4
Data Engineering	3–4
Machine Learning	4
Deep Learning	3–4
LLMs	4
RAG	4
Agentes	3–4
Segurança	3–4
Cloud	3–4
Docker	4
Kubernetes	2–3 inicialmente
MLOps/LLMOps	3–4
System Design	4
Negócios/produto	3–4
Inglês técnico	4

VISÃO GERAL DOS 4 ANOS

ANO 1

- |
- |— Lógica
- |— Python
- |— Algoritmos
- |— Estruturas de dados
- |— Git
- |— Linux
- |— SQL
- |— Engenharia de software
- |— Testes
- |— primeiros sistemas reais

↓

ANO 2

- |
- |— Backend
- |— APIs
- |— PostgreSQL
- |— Data Engineering
- |— Matemática
- |— Estatística
- |— Machine Learning
- |— Modelos clássicos
- |— primeiros projetos profissionais

↓

ANO 3

- |
- |— Deep Learning
- |— PyTorch
- |— NLP
- |— Transformers
- |— LLMs
- |— RAG
- |— AI Engineering
- |— avaliação
- |— agentes
- |— sistemas de IA

↓

ANO 4

- |
- |— Distributed Systems
- |— Docker
- |— Cloud
- |— CI/CD
- |— MLOps
- |— LLM0ps
- |— Observabilidade
- |— Segurança
- |— System Design
- |— Sistemas empresariais
- |— CAPSTONE

Mas isso não significa estudar cada assunto isoladamente.

Matemática, inglês, Git, testes, documentação e engenharia acompanham a formação inteira.

FASE 0 — MENTALIDADE DE ENGENHARIA

Objetivo

Aprender a pensar como engenheiro antes de aprender centenas de ferramentas.

Conteúdo

- decomposição de problemas; abstração; modelagem; causa vs sintoma; requisitos; restrições; hipóteses; trade-offs; experimentação; debugging; documentação; tomada de decisão técnica.

Exercício central

Receber problemas simples como:

“Uma empresa quer automatizar seus relatórios.”

E responder:

- Quem usa? Qual problema? Qual entrada? Qual saída? Qual frequência? Quais dados? Quais restrições? Quanto custa? O que pode dar errado? Como medir sucesso?

Isso começa agora.

FASE 1 — COMPUTAÇÃO + LÓGICA + PYTHON

Meses 1–6

Computação

- bits e bytes; representação de números; representação de texto; arquivos; memória; armazenamento; CPU; processos; terminal; sistema operacional; paths; ambiente; variáveis de ambiente.

Lógica

- proposições; operadores lógicos; condições; decomposição; algoritmos; pseudocódigo; fluxogramas; invariantes; casos de borda.

Python

Do básico até:

- funções; escopo; módulos; packages; exceptions; arquivos; JSON; comprehensions; iterators; generators; decorators; context managers; typing; dataclasses; OOP; composição; herança; polimorfismo; async; profiling; debugging.

Projeto

CLI de gerenciamento empresarial

Por exemplo: clientes, produtos, pedidos, relatórios, logs, persistência, testes.

Não apenas uma calculadora.

FASE 2 — ALGORITMOS + ESTRUTURAS DE DADOS

Meses 3–8

Aqui entra:

- arrays; listas; stacks; queues; hash tables; sets; linked lists; árvores; heaps; grafos; recursão; busca; ordenação; BFS; DFS; shortest path; programação dinâmica básica.

E principalmente: **Complexidade**

- Big-O; tempo; memória; trade-offs; escalabilidade.

Projeto

Construir algumas estruturas manualmente, sem biblioteca pronta. Depois comparar com implementações reais.

Objetivo: saber por que determinada estrutura existe. Não decorar LeetCode.

FASE 3 — ENGENHARIA DE SOFTWARE

Meses 5–12

Aqui começa a transformação de:

“sei programar”

para:

“sei construir software.”

Conteúdo

- Git; GitHub; branches; merge; rebase; pull requests; code review; semantic versioning; documentação; README; testes unitários; integração; E2E; mocking; debugging; logging; exceptions; configuração; dependências; ambientes; packaging.

Design

- modularidade; coesão; acoplamento; SOLID; DRY; KISS; YAGNI; design patterns; refatoração; arquitetura em camadas.

Projeto

Transformar um projeto antigo em software profissional:

`src/`, `tests/`, `docs/`, `config/`, `scripts/`, `README.md`, `pyproject.toml`, CI

■ FASE 4 — LINUX + REDES + SISTEMAS

Meses 6–14

Linux

- filesystem; permissions; processes; signals; pipes; shell; environment; SSH; package management; system services; logs.

Redes

- IP; TCP; UDP; ports; DNS; HTTP; HTTPS; TLS; sockets; proxies; load balancers; latency; bandwidth.

Projeto

Construir uma aplicação cliente-servidor simples e observar o tráfego.

Objetivo: entender o que acontece abaixo do framework.

■ FASE 5 — SQL + DATABASE ENGINEERING

Meses 7–15

PostgreSQL será o banco principal. A documentação atual mantém PostgreSQL 18 como versão corrente, enquanto a linha 19 está em beta, então o roadmap deve ensinar conceitos estáveis em vez de amarrar sua formação a uma versão específica. PostgreSQL

SQL

- SELECT; WHERE; JOIN; GROUP BY; HAVING; subqueries; CTE; window functions; views; constraints; indexes; transactions.

Internals

- B-tree; query planner; locks; isolation; ACID; concurrency; normalization; denormalization.

Outros bancos

Conhecer: Redis; document databases; key-value; graph; vector databases.

Mas sempre responder: “Por que não PostgreSQL?” antes de escolher outro.

Projeto

Mini ERP: usuários, clientes, produtos, estoque, pedidos, pagamentos, auditoria, relatórios.

Esse será um dos primeiros grandes projetos do portfólio.

FASE 6 — DATA ENGINEERING + DATA ANALYTICS

Meses 9–18

Aqui entra o que você marcou como Data Analytics, mas como competência de Engenharia de IA, não como carreira paralela.

Data Analytics

- métricas; KPIs; agregações; análise exploratória; visualização; correlação; distribuição; outliers; hipóteses; interpretação; comunicação de resultados.

A pergunta central: “O que os dados realmente estão dizendo?”

Data Engineering

- ETL; ELT; pipelines; batch; streaming; schemas; validação; data quality; lineage; data warehouse; data lake; object storage; particionamento; feature engineering.

Projeto

Pipeline:

API ↓ Raw data ↓ Validation ↓ Transformation ↓ PostgreSQL/Warehouse ↓ Analytics ↓ ML dataset

Isso conecta dados → análise → IA.

FASE 7 — MATEMÁTICA PARA IA

Meses 8–24, paralelamente

Não estudar matemática como uma matéria isolada. Sempre: matemática ↓ intuição ↓ implementação ↓ aplicação em IA

- **Álgebra:** funções; exponenciais; logaritmos; equações; sistemas.
- **Matemática discreta:** lógica; conjuntos; relações; funções; combinatória; grafos; indução.
- **Álgebra linear:** vetores; matrizes; espaços vetoriais; base; dimensão; transformações; produto interno; norma; projeção; determinante; posto; autovalores; autovetores; SVD; PCA.
- **Cálculo:** limites; derivadas; derivadas parciais; gradiente; regra da cadeia; Jacobiana; Hessiana; otimização; integrais relevantes.
- **Probabilidade:** eventos; probabilidade condicional; Bayes; variáveis aleatórias; distribuições; esperança; variância; covariância.
- **Estatística:** amostragem; inferência; testes; máxima verossimilhança; intervalos de confiança; avaliação experimental.
- **Informação:** entropia; cross-entropy; KL divergence; informação mútua.

Objetivo: Não virar matemático. Virar engenheiro que entende o que o modelo está fazendo matematicamente.

FASE 8 — MACHINE LEARNING

Meses 12–20

Antes de Deep Learning.

Fundamentos

- supervised; unsupervised; reinforcement learning; features; labels; preprocessing; normalization; standardization; train/validation/test; cross-validation; leakage; overfitting; underfitting; bias/variance; regularização; feature engineering; hyperparameter tuning; error analysis.

Modelos (Implementar e entender)

- linear regression; logistic regression; k-NN; decision tree; random forest; gradient boosting; XGBoost; SVM; k-means; PCA; anomaly detection.

Métricas

- accuracy; precision; recall; F1; ROC-AUC; PR-AUC; confusion matrix; MAE; MSE; RMSE; R^2 .

Projeto

Sistema de previsão empresarial

Exemplo: dados históricos ↓ pipeline ↓ features ↓ treinamento ↓ avaliação ↓ API ↓ dashboard

FASE 9 — DEEP LEARNING

Meses 17–25

PyTorch será o framework principal. O framework é uma ferramenta; o objetivo é compreender tensores, autograd, treinamento, arquitetura e performance — não decorar APIs.

Fundamentos

- tensor; forward pass; loss; gradient; backpropagation; gradient descent; optimizer; batch; epoch; learning rate; regularização; dropout; normalization; scheduling.

Arquiteturas

- perceptron; MLP; CNN; RNN; LSTM; GRU; autoencoder; attention; Transformer.

Projeto

Treinar um modelo de verdade e produzir:

dataset ↓ experimentos ↓ training ↓ validation ↓ error analysis ↓ model artifact ↓ inference API

■ FASE 10 — NLP + TRANSFORMERS

Meses 20–27

Entender profundamente:

- tokenização; vocabulary; embeddings; positional encoding; attention; self-attention; Q/K/V; multi-head attention; encoder; decoder; masking; causal modeling; contexto.

Aqui o estudante deverá conseguir explicar: por que Transformer funciona e como ele produz a base arquitetural dos LLMs.

■ FASE 11 — LLM ENGINEERING

Meses 22–30

Agora entra GenAI profissional.

Conceitos

- pretraining; instruction tuning; SFT; RLHF; preference optimization; inference; sampling; temperature; top-k; top-p; context window; hallucination; grounding; multimodalidade; latency; token economics.

APIs

- structured output; streaming; function calling; tool calling; timeout; retries; rate limits; fallback; caching; model routing.

A meta deixa de ser: “fiz um chatbot.”

E passa a ser: “construí uma aplicação confiável que usa modelos.”

■ FASE 12 — RAG

Meses 23–31

Arquitetura:

documentos ↓ ingestion ↓ parsing ↓ chunking ↓ metadata ↓ embeddings ↓ storage ↓ retrieval ↓ reranking ↓ context ↓ LLM ↓ response ↓ evaluation

Dominar:

- chunking; overlap; metadata; vector search; dense retrieval; BM25; hybrid search; reranking; query rewriting; multi-query; contextual retrieval; citations; grounding; retrieval evaluation; RAG evaluation; atualização da base.

Projeto

RAG empresarial

Exemplo: sistema que responde perguntas sobre documentação interna.

Com: autenticação; permissões; citations; avaliação; logging; tracing; métricas.

FASE 13 — AI AGENTS

Meses 25–32

Somente depois de dominar workflows.

Fundamentos

- agent loop; state; memory; tools; planning; function calling; orchestration; workflows; human-in-the-loop; guardrails; retries; failure handling; permissions; observability; evaluation.

A distinção obrigatória:

Workflow: $A \rightarrow B \rightarrow C \rightarrow D$

Agent: objetivo ↓ observação ↓ decisão ↓ ação ↓ observação ↓ nova decisão

O estudante precisa saber justificar quando NÃO usar agente. Isso é Nível 4.

FASE 14 — FINE-TUNING

Meses 26–32

Somente quando houver justificativa.

- dataset preparation; instruction datasets; SFT; LoRA; PEFT; quantization; QLoRA; evaluation; overfitting; catastrophic forgetting.

E principalmente:

Prompting vs RAG vs Fine-tuning

A decisão deverá partir dos requisitos.

FASE 15 — BACKEND PROFISSIONAL

Meses 15–30, progressivamente

FastAPI é uma excelente escolha para a formação porque a documentação atual cobre APIs, OpenAPI, testes, WebSockets, webhooks, deployment, Docker e tópicos avançados. FastAPI +1

Dominar:

- HTTP; REST; OpenAPI; schemas; Pydantic; validação; autenticação; autorização; JWT; OAuth; pagination; filtering; rate limiting; caching; webhooks; queues; background jobs; WebSockets; database integration.

Projeto

SaaS empresarial

Com: frontend ↓ API ↓ auth ↓ PostgreSQL ↓ background jobs ↓ cache ↓ AI service

FASE 16 — DOCKER + DEPLOYMENT

Meses 25–34

Docker entra depois de entender sistemas.

- containers; images; layers; Dockerfile; volumes; networks; Compose; registry; secrets; multi-stage builds.

A documentação atual do Docker estrutura exatamente esses fundamentos: containers, images, registries e Compose. Docker Documentation +1

Projeto

Pegar uma aplicação anterior e: local ↓ Docker ↓ CI ↓ staging ↓ production

FASE 17 — CLOUD

Meses 28–36

Escolher uma cloud principal. A formação deve dominar conceitos antes de decorar serviços.

Fundamentos

- compute; storage; networking; IAM; databases; queues; object storage; containers; serverless; GPUs; monitoring; cost management.

Segurança entra junto. IAM, por exemplo, não é “configuração depois”: permissões e acesso a recursos precisam fazer parte do desenho do sistema desde o início. Documentação AWS

FASE 18 — DEVOPS + CI/CD

Meses 28–37

- CI; CD; pipelines; testing; build; artifacts; staging; production; deployment; rollback; secrets; environment management.

FASE 19 — KUBERNETES

Meses 32–38

Somente agora.

- Pods; Deployments; Services; Ingress; ConfigMaps; Secrets; health checks; resource limits; scaling; rolling updates; networking; storage.

Objetivo: entender orquestração. Não: memorizar 300 comandos kubectl.

FASE 20 — MLOps + LLMOps

Meses 28–40

Aqui o estudante passa de: “modelo funciona” para: “modelo é operacionalmente sustentável.”

MLOps:

- experiment tracking; datasets; data versioning; model versioning; model registry; training pipelines; deployment; monitoring; drift; retraining.

LLMOps:

- prompt versioning; evaluation datasets; tracing; evaluation; token monitoring; cost; latency; regression tests; model routing; production traces.

MLflow atualmente oferece justamente tracking, model registry, deployment, avaliação, tracing e capacidades para aplicações LLM/agentes. MLflow AI Platform +2

FASE 21 — OBSERVABILIDADE

Meses 30–40

Dominar: Logs, Metrics, Traces

Além de:

- health checks; alerts; correlation IDs; latency; throughput; error rate; token usage; cost; model quality; retrieval quality.

OpenTelemetry é particularmente importante como fundamento porque é vendor-neutral e trabalha justamente com geração, coleta e exportação de traces, metrics e logs. OpenTelemetry +1

FASE 22 — SEGURANÇA DE IA

Transversal + aprofundamento meses 28–40

Não será uma disciplina isolada. Segurança deverá aparecer desde o primeiro backend.

Software:

- authentication; authorization; TLS; secrets; encryption; secure coding; dependency security; supply chain; OWASP.

IA:

- prompt injection; indirect prompt injection; sensitive information disclosure; output handling; data/model poisoning; excessive agency; system prompt leakage; vector/embedding weaknesses; misinformation; unbounded consumption.

Esses riscos aparecem explicitamente no OWASP Top 10 para LLM Applications 2025. OWASP Gen AI Security Project +2

FASE 23 — SISTEMAS DISTRIBUÍDOS

Meses 30–42

- CAP; consistency; availability; partition tolerance; replication; sharding; caching; queues; message brokers; pub/sub; eventual consistency; distributed transactions; retries; circuit breakers; backpressure; fault tolerance.

Aqui começa a verdadeira arquitetura de sistemas.

FASE 24 — SYSTEM DESIGN

Meses 32–44

Aprender a transformar:

problema ↓ requisitos funcionais ↓ requisitos não funcionais ↓ estimativas ↓ arquitetura ↓ componentes ↓ dados ↓ infraestrutura ↓ trade-offs

Sempre considerar:

- throughput; latency; availability; scalability; reliability; consistency; security; cost; maintainability; observability.

Exercícios

Projetar: WhatsApp simplificado; sistema de pagamentos; plataforma de documentos; sistema de recomendação; RAG empresarial; plataforma de agentes; pipeline de ML; SaaS multi-tenant.

FASE 25 — SISTEMAS EMPRESARIAIS

Aqui acontece a integração brutal de tudo.

Construir progressivamente:

- Sistema 1: CRM.
- Sistema 2: ERP simplificado.
- Sistema 3: Sistema de estoque.
- Sistema 4: Workflow engine.
- Sistema 5: Helpdesk.
- Sistema 6: Document management.
- Sistema 7: Dashboard empresarial.
- Sistema 8: Sistema multi-tenant.
- Sistema 9: Billing.
- Sistema 10: Plataforma de automação.
- Sistema 11: Copiloto empresarial.
- Sistema 12: RAG corporativo.
- Sistema 13: Agente empresarial.

Esses não precisam ser 13 projetos gigantes. Vários podem evoluir o mesmo sistema.

FASE 26 — AUTOMAÇÃO

Transversal

- arquivos; PDF; Excel; documentos; e-mail; navegador; APIs; scraping ético; jobs; queues; workflows; RPA; human-in-the-loop.

Regra: API primeiro; automação de interface apenas quando necessário.

FASE 27 — PERFORMANCE

Transversal + aprofundamento

- profiling; caching; batching; async; concurrency; multiprocessing; vectorization; GPU; quantization; model optimization; latency optimization; throughput optimization; cost optimization.

■ FASE 28 — NEGÓCIOS E PRODUTO

Isso é importantíssimo para o objetivo que você definiu.

Aprender:

- descoberta de problemas; requisitos; stakeholders; ROI; custo; pricing; MVP; SaaS; suporte; manutenção; produto; consultoria; empreendedorismo.

Toda arquitetura deve responder: “Quanto valor isso gera?” e não simplesmente: “Qual framework novo posso usar?”

■ FASE 29 — INGLÊS TÉCNICO

Durante os 4 anos.

Gradualmente:

- **Ano 1:** documentação; vocabulário; Stack Overflow/GitHub; README.
- **Ano 2:** escrita técnica; issues; pull requests; documentação de APIs.
- **Ano 3:** apresentações; comunicação técnica; reuniões.
- **Ano 4:** entrevistas; system design interviews; comunicação profissional.

A evolução deverá ser:

**Lógica ↓ Scripts ↓ Automação ↓ CLI ↓ Sistemas ↓ SQL ↓ APIs ↓ Backend ↓ ML ↓ Deep Learning ↓
LLM ↓ RAG ↓ Agents ↓ Sistemas empresariais ↓ Produção**

O GitHub não deve ser um depósito de códigos. Cada projeto relevante terá:

```
README
|
├─ problema
├─ requisitos
├─ arquitetura
├─ decisões
├─ trade-offs
├─ implementação
├─ testes
├─ segurança
├─ observabilidade
├─ deployment
├─ métricas
├─ custos
├─ limitações
└─ próximos passos
```

PROJETO CAPSTONE

Nos últimos meses:

Sistema Empresarial Completo de IA

Exemplo conceitual: AI Operations Platform

Uma empresa fornece: documentos; tickets; dados; processos; usuários.

O sistema:

```
Usuário
↓
Frontend
↓
API Gateway
↓
Auth/RBAC
↓
Application Layer
↓
Workflow
├─ RAG
├─ LLM
├─ Tools
└─ Agents
↓
Data Layer
├─ PostgreSQL
├─ Object Storage
├─ Redis
└─ Vector Search
↓
Evaluation
↓
Observability
↓
Cloud
↓
Production
```

Com:

multi-tenancy; RBAC; auditoria; RAG; agentes; avaliação; tracing; métricas; segurança; CI/CD; Docker; cloud; monitoramento; documentação; análise de custos.

Esse projeto deve parecer um produto profissional, não um trabalho escolar.

Período	Foco dominante
Meses 1–3	Lógica + Python
Meses 4–6	Python + algoritmos
Meses 7–9	Git + software + Linux
Meses 10–12	SQL + PostgreSQL
Meses 13–15	Backend + APIs
Meses 16–18	Data Engineering + matemática
Meses 19–21	Machine Learning
Meses 22–24	ML + Deep Learning
Meses 25–27	PyTorch + Transformers
Meses 28–30	LLM Engineering
Meses 31–33	RAG + avaliação
Meses 34–36	Agents + AI Engineering
Meses 37–39	Docker + Cloud + MLOps
Meses 40–42	Segurança + Observabilidade
Meses 43–45	Distributed Systems + System Design
Meses 46–48	Capstone + carreira

Inglês, Git, testes, documentação e matemática continuam durante todo o período.

 VERSÃO ACELERADA — 3 ANOS

Se for necessário comprimir:

- **Ano 1:** Python + Computação + Algoritmos + Git + Linux + SQL + Backend + Matemática
- **Ano 2:** Data Engineering + ML + Deep Learning + PyTorch + NLP + Transformers + LLMs + RAG
- **Ano 3:** AI Engineering + Agents + MLOps + LLMOps + Docker + Cloud + Security + Observability + System Design + Capstone

A versão de 4 anos é preferível porque permite levar mais fundamentos ao Nível 4 sem transformar o estudo em corrida.

GRAFO DE DEPENDÊNCIAS

```
LÓGICA
↓
PYTHON
↓
ALGORITMOS —————> ESTRUTURAS DE DADOS
↓                      ↓
ENGENHARIA DE SOFTWARE ←————
↓
GIT + TESTES + DEBUGGING
↓
LINUX + REDES
↓
SQL + DATABASES
↓
BACKEND + APIs
↓
DATA ENGINEERING
↓
MATEMÁTICA + ESTATÍSTICA
↓
MACHINE LEARNING
↓
DEEP LEARNING
↓
TRANSFORMERS
↓
LLMs
|—————> RAG
|          ↓
|—————> AGENTS
↓
AI ENGINEERING
↓
MLOps / LLMops
↓
Docker / Cloud / CI/CD
↓
Observability / Security
↓
Distributed Systems
↓
System Design
↓
Enterprise AI Systems
↓
CAPSTONE
```

E há dois eixos atravessando tudo: MATEMÁTICA e INGLÊS

PRIORIDADE

Essencial — Nível 4

- lógica; Python; algoritmos; estruturas de dados; Git; engenharia de software; SQL; bancos; APIs; backend; fundamentos de sistemas; data engineering; ML; avaliação; LLM engineering; RAG; AI architecture; segurança; observabilidade; system design.

Importante — Nível 3–4

- Deep Learning; PyTorch; Transformers; agents; MLOps; LLMOps; Docker; cloud; distributed systems; performance; produto.

Especialização — Nível 2–3 inicialmente

- Kubernetes avançado; multi-agent systems; fine-tuning avançado; distributed training; GPU optimization profunda; modelos multimodais avançados; sistemas de inferência de larga escala.

Isso evita um erro clássico: tentar virar simultaneamente AI Engineer + ML Researcher + Data Engineer + DevOps Specialist + Cloud Architect em quatro anos.

AUDITORIA DE LACUNAS

Fiz a auditoria conceitual do roadmap original. E encontrei alguns pontos que precisavam ser explicitados.

1. Avaliação

Seu prompt já mencionava avaliação, mas ela precisava virar disciplina transversal. Agora existe: software evaluation, ML evaluation, LLM evaluation, RAG evaluation, agent evaluation, production evaluation. Isso é extremamente importante. MLflow, por exemplo, atualmente enfatiza avaliação contínua e avaliação de traces de aplicações em produção. MLflow AI Platform +1

2. Observabilidade

Não deve ser “aprender logs no final”. Deve começar em software e evoluir até IA.

3. Segurança

Também não deve aparecer somente depois de agentes. Ela começa com: auth, permissions, secrets, input validation e evolui para: prompt injection, agent security, data leakage, excessive agency.

4. Data Analytics

Foi incorporado como competência intermediária: dados → análise → decisão → ML em vez de virar uma segunda carreira.

5. Produto

Foi incorporado porque Engenharia de IA empresarial sem compreensão do problema de negócio vira apenas engenharia de tecnologia.

6. System Design

Foi movido para o final porque precisa de experiência acumulada.

7. Kubernetes

Foi deliberadamente empurrado para depois de Docker, Cloud e infraestrutura.

8. Agentes

Foram empurrados para depois de workflows, APIs, LLMs, RAG, avaliação e segurança. Isso reduz muito o risco de aprender “agentes” como uma coleção de frameworks.



A REGRA MAIS IMPORTANTE DO ROADMAP

Para cada tecnologia, o Gemini deverá perguntar:

1. Por que isso existe?
2. Que problema resolve?
3. Como funciona por baixo?
4. Como implementar?
5. Como testar?
6. Como quebrar?
7. Como monitorar?
8. Quando não usar?
9. Qual alternativa?
10. Qual trade-off?
11. Quanto custa?
12. Como evolui em produção?

Se você consegue responder apenas às primeiras três: **Conhecer/Explicar**.

Se consegue construir: **Implementar**.

Se consegue escolher entre alternativas: **Projetar**.

O QUE EU CONSIDERO A SAÍDA IDEAL

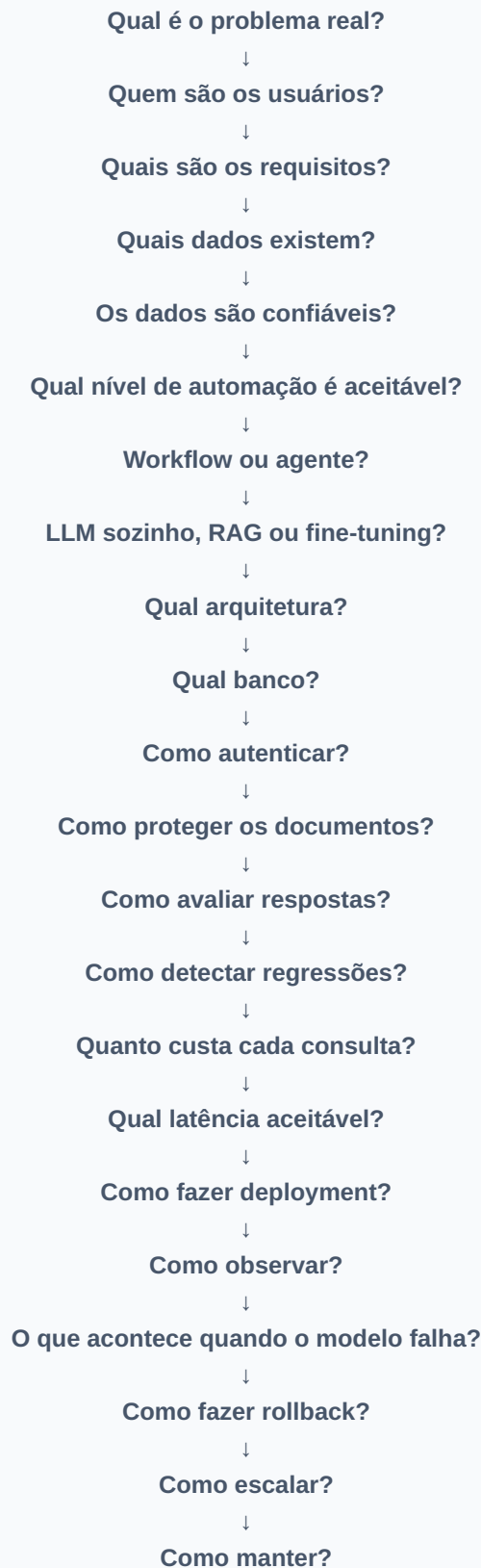
No final dos 4 anos, diante de:

“Uma empresa quer automatizar o atendimento interno usando IA.”

Você não deveria responder:

“Vou fazer um chatbot com RAG.”

Você deveria pensar:



É isso que diferencia um usuário de IA de um Engenheiro de IA.

E essa direção é coerente com o ecossistema atual: FastAPI cobre construção e testes de APIs; Docker trata empacotamento e execução consistente; Kubernetes trata orquestração; OpenTelemetry fornece uma camada agnóstica de observabilidade; e MLflow já cobre partes importantes de tracking, avaliação, deployment e observabilidade de ML/LLMs/agentes.